

Polyhedral-Based Compilation Framework for In-Memory Neural Network Accelerators

JIANHUI HAN, XIANG FEI, ZHAOLIN LI, and YOUHUI ZHANG, Tsinghua University, China

Memristor-based processing-in-memory architecture is a promising solution to the memory bottleneck in the **neural network** (NN) processing. A major challenge for the programmability of such architectures is the automatic compilation of high-level NN workloads, from various operators to the memristor-based hardware that may provide programming interfaces with different granularities. This article proposes a source-to-source compilation framework for such memristor-based NN accelerators, which can conduct automatic detection and mapping of multiple NN operators based on the flexible and rich representation capability of the polyhedral model. In contrast to previous studies, it implements support for pipeline generation to exploit the parallelism in the NN loads to leverage hardware resources for higher efficiency. The evaluation based on synthetic kernels and NN benchmarks demonstrates that the proposed framework can reliably detect and map the target operators. Case studies on typical memristor-based architectures also show its generality over various architectural designs. The evaluation further demonstrates that compared with existing polyhedral-based compilation frameworks that do not support the pipelined execution, the performance can upgrade by an order of magnitude with the pipelined execution, which emphasizes the necessity of our improvement.

CCS Concepts: • **Hardware** → **Emerging technologies**; • **Computer systems organization** → **Neural networks**;

Additional Key Words and Phrases: Polyhedral model, memristor, processing-in-memory

ACM Reference format:

Jianhui Han, Xiang Fei, Zhaolin Li, and Youhui Zhang. 2021. Polyhedral-Based Compilation Framework for In-Memory Neural Network Accelerators. *J. Emerg. Technol. Comput. Syst.* 18, 1, Article 15 (September 2021), 23 pages.

<https://doi.org/10.1145/3469847>

1 INTRODUCTION

As **neural networks** (NNs) achieve state-of-the-art performance in a wide range of AI applications [3, 20, 34, 36], there have been increasing studies on the design of efficient NN accelerators.

This work was supported in part by National Key Research and Development Program of China under Grant No. 2020YFB1600200, in part by National Natural Science Foundation of China (NSFC) under Grant No. 62072266 and Grant No. 62050340, in part by Beijing Academy of Artificial Intelligence (BAAI) under Grant No. BAAI2019ZD0403, and in part by Beijing Innovation Center for Future Chip (ICFC).

Authors' addresses: J. Han, School of Integrated Circuits, Beijing National Research Center for Information Science and Technology, Tsinghua University, 30 Shuangqing Rd., Haidian, Beijing 100084, China; email: hanjh16@mails.tsinghua.edu.cn; X. Fei, Z. Li, and Y. Zhang, Department of Computer Science and Technology, Beijing National Research Center for Information Science and Technology, Tsinghua University, 30 Shuangqing Rd., Haidian, Beijing 100084, China; emails: feix16@mails.tsinghua.edu.cn, lzl73@mail.tsinghua.edu.cn, zyh02@tsinghua.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2021 Association for Computing Machinery.

1550-4832/2021/09-ART15 \$15.00

<https://doi.org/10.1145/3469847>

Among these accelerators, memristor-based architectures [5, 12, 33, 35] have been proven to be superior to their counterparts based on conventional CMOS-based digital circuits, as they are highly efficient at performing in-memory matrix multiplication. One reason is that memristor devices feature low computing latency and high area efficiency. For example, the resistive-capacitive delay is approximately 10 ps for a **resistive random access memory (ReRAM)** crossbar with the size of 100×100 [43], whereas the area of a ReRAM cell is approximately $4F^2$ [14], where F is the feature size of the integrated circuit process. Furthermore, their in-memory processing manner reduces the data movement overhead significantly, which is beneficial for both performance and energy saving.

One key factor to fully exploit such memristor-based accelerators is how to efficiently detect, extract, and map NN operators onto the accelerators. A commonly adopted programming paradigm [12, 35] provides **application programming interface (API)** functions to improve the programmability of such architectures. However, much work is still left to programmers with a such programming paradigm. They are required to have detailed knowledge about both the hardware architecture and the software workloads and decide the operators to offload. As this workflow hinders the exploitation of memristor-based accelerators, an automatic compilation framework is needed.

Recently, there have been studies that proposed compilation frameworks where the polyhedral model is utilized to automatically detect and offload NN operators onto memristor crossbars [15, 37]. However, these studies still leave several crucial problems unsolved.

First, they focus on detecting **matrix-matrix multiplication (MM)** and **matrix-vector multiplication (MV)**, whereas there are other primary operators in NN workloads. For example, **convolution (CONV)** is an essential building operator of **convolutional neural networks (CNNs)** and cannot be directly viewed as normal MM. Although TDO-CIM [37] was tested on a benchmark that contains CONV operators, it still lacks proper reshaping of CONV operators to improve their execution efficiency on crossbars. Such reshaping strategies have been well investigated [13, 33, 35] and should be implemented in the compilation framework. It requires not only changing the scheduling order but also transforming the data layout in memory.

Another example is activation functions and fused operators. Memristor-based NN accelerators often implement various activation functions [33, 35], which requires that fused operators (e.g., MM+ReLU) should be considered as a whole. Some memristor-based accelerators provide co-located MM and activation units [12, 33] to reduce the amount of data movement, and others directly implement fused operators to benefit from the efficient peripheral circuit design [18, 24]. In such cases, fused operators take priority over the non-fused ones when offloading.

Second, since TC-CIM [15] and TDO-CIM [37] are both built based on Loop Tactics [8] and adopt a *detect-and-rebuild* manner where the schedule tree is modified right after a target operator is found, they conduct only the operator-level offloading and ignore the opportunity to optimize the NN workload as a whole. As memristor devices have high area efficiency, a large number of **processing elements (PEs)** are provided. When mapping multiple operators onto such accelerators, optimized resource allocation among the operators should be decided by the compilation framework. Based on that, pipelines can be used to improve the utilization of the PEs and the overall throughput [24, 32, 35]. According to our evaluation, failing to take advantage of the pipelined execution can lead to performance loss by an order of magnitude (details are revealed in Section 4.4.5). However, these functions require an overview of the whole NN workload, which is not implemented by these previous studies.

Therefore, this article proposes a polyhedral-model-based source-to-source NN compilation framework for memristor-based in-memory accelerators. With the flexibility and representation capability of the polyhedral model, we extend the detection from MV/MM to a set of operators,

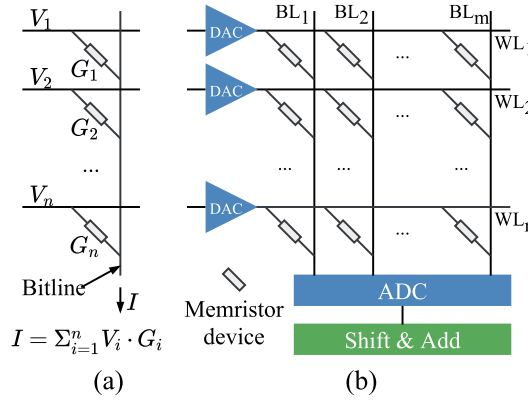


Fig. 1. (a) Dot-product calculation with memristor crossbar. (b) Memristor-based architecture for matrix multiplication.

including MV, MM, CONV, and MM/CONV-centric fused operators. The framework also supports the generation of pipelined execution code for the whole NN workload. At the code generation phase, different *invocation granularities* are supported, where coarse-grained invocation is used to support integration with native toolchains of the accelerators.

The contributions of this article are summarized as follows.

- We propose a source-to-source NN compilation framework, which automatically detects and maps various types of NN operators onto memristor-based accelerators.
- We implement the supportive functionality to facilitate pipeline generation, including the resource allocation between operators and the generation of pipelined execution codes. The generated pipeline can exploit the *batch* and *layer* dimensions of NN workloads to increase the processing throughput.
- We conduct experiments with two memristor-based architectures, ISAAC [33] and FPSA [24], to evaluate the different invocation granularity of the compilation framework. Furthermore, the evaluation demonstrates that compared with existing polyhedral-based compilation frameworks that do not support the pipelined execution, the performance can upgrade by an order of magnitude with the pipelined execution.

2 BACKGROUND AND RELATED WORK

2.1 Memristor-Based NN Acceleration

NN applications are both memory-intensive and computation-intensive. Therefore, there are many NN accelerators [9, 10, 19, 25, 46] based on mature digital circuits to speed up NN processing. To further boost the performance and eliminate the *memory wall* problem, memristor-based NN accelerators have also been proposed.

Memristor is a type of emerging non-volatile memory, which stores information with its conductance. Prior work [21] has shown that the memristor-based crossbar is very efficient at performing analog MV computation in the location where the matrices are stored. As shown in Figure 1(a), the **bitline** (BL) is connected to each **wordline** (WL) via one memristor device. The conductance of the memristor devices is modulated to $G_1, G_2, \dots, G_n$, and the current of each device is the multiplication of applied voltage V_i and G_i . The dot product of two vectors is the accumulated currents at BL by Kirchhoff's law.

Several existing studies [5, 12, 22, 24, 32, 33, 35] proposed to use memristor crossbar array to accelerate NN applications. The commonly used architecture is shown in Figure 1(b), where the memristor crossbar stores the weight parameters and performs the MV computation along with its WL/BL peripherals. The input vectors are converted into analog signals by **digital-to-analog converters (DACs)** and fed into the memristor crossbar, and the output signals at each BL are converted back to digital signals by **analog-to-digital converters (ADCs)**. Usually, multiple memristor cells are concatenated to represent one weight parameter due to the limited number of device conductance levels. Shift-and-add units are used to accumulate the results from different columns to reform the high-precision results. For example, ISAAC [33] conservatively uses eight cells to represent one 16-bit cell; each cell represents 2 bits. Other functional units are also implemented in hardware to accelerate the whole NN model, such as activation units, pooling units, and so forth. Based on the PEs with such architectures, accelerators are built by connecting the PEs with proper on-chip interconnection, such as network-on-chip or hierarchical memory bus.

2.2 Software for Memristor-Based Architectures

Several previous studies proposed ISA for memristor-based architectures and built software tools based on that. Fujiki et al. [16] proposed a programmable in-memory data-parallel processor, which relies on the instruction set to provide generalized computation capabilities. A compilation framework that can convert TensorFlow [1] dataflow graph into its low-level instructions was also developed. Ankit et al. [5] proposed the runtime compiler along with the architecture design of PUMA. It is a runtime compiler implemented as a C++ library for programmers. During compiling, the model is lowered to the assembly code of PUMA's ISA. Ambrosi et al. [2] proposed an end-to-end software stack to support ISA-programmable memristor-based accelerators, aiming to improve the programmability issue. It relies on the **Open Neural Network Exchange (ONNX)** standard [30] to leverage the interoperable NN models developed on NN frameworks. Although the introduction of ISA facilitates fine-grained programmability, it requires the implementation of hardware decoders, which can lower the high area efficiency of memristor crossbars.

Therefore, this article focuses on the architectures where the accelerators expose an API for the programmer, and proper workloads are offloaded from the host to the accelerator by API invocation with various levels. Such an approach has been adopted by several architectural designs [12, 26, 35], as well as implementation based on real memristor chips [44]. For example, PipeLayer [35] defines a set of API functions to perform input/output data movement, weight loading, and accelerator control. However, it is still the responsibility of programmers to write the application with the given API. In our work, the proposed compilation framework aims to transparently handle the API invocation to improve the programmability of memristor-based NN accelerators.

There are also some end-to-end frameworks for memristor-based architectures [27, 45]. These works focus on the device and circuit-level factors, such as device noise and weight quantization, which are often tightly coupled with specific hardware designs. Based on these factors, the conversion from high-precision weight parameters to low-precision device conductance and the physical mapping of the weights are discussed in detail. However, they lack the capability of automatically detecting and mapping operators, which is the key functionality for our proposed framework to improve the programmability. This limits the generality of such frameworks.

2.3 Polyhedral Model and Schedule Tree

The polyhedral model provides a powerful representation of loop nests and the transformations on them. It has been integrated into several mature compilers [17, 31], and there are also studies that proposed standalone polyhedral-based compilers [7, 42]. In recent years, it has also been widely

studied for generating efficient code that targets both general-purpose processors and domain-specific accelerators [6, 29, 38].

Within the polyhedral model, the computations to be performed are described by *iteration domain*, which is a set of statement instances inside loop nests. Based on the iteration domain, the *schedule* assigns the execution order to each instance in the iteration domain. A *schedule tree* [41] is a representation of affine schedules, which is designed to eliminate representation redundancy, improve maintainability, and enable extensions. In this subsection, we provide a brief introduction to the schedule tree components that are relevant to the further discussion.

A schedule tree consists of multiple tree nodes, which belong to several types: *domain nodes*, *sequence nodes*, *set nodes*, *filter nodes*, *band nodes*, *context nodes*, *extension nodes*, and *mark nodes*. In addition, there are *leaf nodes* that appear as the leaves of the schedule tree.

Domain nodes denote the iteration domain of the whole schedule tree and should be the root of the schedule tree. *Sequence nodes* and *set nodes* encode the scheduling order represented by the schedule tree. The former indicates that their children are executed sequentially, whereas the latter encodes concurrency, indicating that their children can be executed in any order or even concurrently. Sequence nodes and set nodes can have multiple children in the schedule tree, and their children are always *filter nodes*, which restricts the schedule domain of their subtree to be a subset of the iteration domain. *Band nodes* encode partial affine schedules, which also have properties to indicate whether the band dimensions are permutable or parallel. *Mark nodes* contain annotations of its subtree with arbitrary additional information preserved for code generation. In this article, we use mark nodes to pass on essential information through different phases of the workflow. Details are discussed in Section 3.

Figure 2 is a schedule tree example corresponding to the loop nest code in Listing 1. The band node that is the child of the root domain node represents outer loop nests shared by statements S1 and S2. The sequence node further indicates the execution order of the statements, i.e., the instance represented by S1(*i*, *j*) should be executed before those represented by S2(*i*, *j*, *k*). Then, final band node encodes the schedule for the innermost loop surrounding S2. All leaf nodes are omitted in the figure for clarity.

2.4 Polyhedral Compilation for Memristor-Based Architectures

Although mapping strategies from NNs to memristor-based architectures have been proposed [13, 33, 35, 47], there is very limited research on tools for the compilation of NN workloads for such architectures, which are capable of performing operator detection and mapping automatically. TC-CIM [15] is such a polyhedral compilation framework based on Loop Tactics [8] and **Tensor Comprehensions (TCs)** [38], which aims to enable users to exploit memristor-based in-memory acceleration transparently. TC-CIM first detects and extracts the target operators by recognizing their specific patterns in schedule trees and then rebuilds a matched subtree. The matching process is conducted by *Schedule Tree Matcher* and *Access Relation Matcher* of Loop Tactics. The former is a declarative description of a subtree pattern of the target operator, whereas the latter allows the user to identify whether the memory accesses have desired properties. TDO-CIM [37] is another such compilation framework, which takes C code as its input. It also proposed a lightweight runtime library that includes memory allocation, data transfer, and the execution of computational tasks.

However, as mentioned in Section 1, TC-CIM and TDO-CIM employ matchers targeting MM and MV computation and do not implement dedicated matching and reshaping of CONVs, which loses corresponding optimization opportunities. More importantly, they process NN operators one

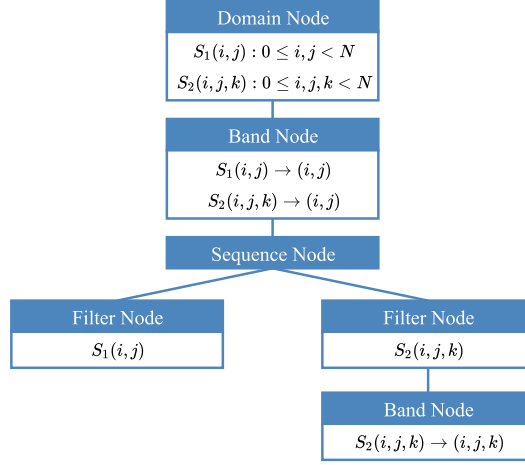


Fig. 2. Schedule tree of code in Listing 1.

```

for (int i=0; i<N; i++)
  for (int j=0; j<N; j++) {
S1:    C[i][j] = 0.0;
        for (int k=0; k<N; k++)
S2:    C[i][j] += A[i][k] * B[k][j];
  }

```

Listing 1. Example code of matrix-matrix multiplication.

by one and modify the schedule tree right after the detection of each operator. When rebuilding one operator, the information about other operators (which are not detected yet) cannot be used for optimization. Therefore, these studies lack consideration for the overall optimization of NN models, such as the resource allocation among all the operators and the generation of pipelines. This motivates us to propose the polyhedral-based compilation framework in this article. Instead of focusing on operator-level offloading, the proposed framework conducts operator detection to capture all the operators of interest and rebuilds the output from the network perspective. In the next section, we will give detailed descriptions of the proposed compilation framework.

3 COMPILATION FRAMEWORK

In this section, we first present an overview of the proposed compilation framework, followed by a description of its components. Then we describe the detailed implementation to support pipelined execution of NN workloads, including hardware resource allocation, pipeline code generation, and runtime support required.

3.1 Overview

Figure 3 gives an overview of the proposed source-to-source compilation framework. The first step is to extract the polyhedral **intermediate representation (IR)** of the original source code. Then the main body of the proposed framework operates on this polyhedral IR. Specifically, the target operators are first automatically detected by running several passes of schedule tree matching. When a match is found, a mark node is inserted into the schedule tree, with all the information needed to build the function calls. After all the matching processes are finished, the actual building

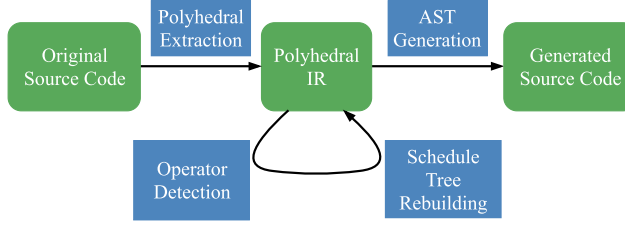


Fig. 3. Overview of the proposed compilation framework.

of the function calls is performed. Finally, the polyhedral IR is used to generate the output source code.

We implement the proposed compilation framework based on *isl* [39] and *pet* [40]. We did not use Loop Tactics [8] to implement the proposed framework, since we find that describing the output source code can be complicated with the tree builder in Loop Tactics, especially when generating the pipelined execution code, as described in Section 3.4. Therefore, the **abstract syntax tree (AST)** node is built directly in the schedule tree rebuilding phase of the proposed framework.

We use the commonly adopted C language to compose both the original and the generated source code due to the native support of the polyhedral libraries. Although using C language to describe NN workloads can be inefficient compared with popular machine learning frameworks, the core idea of detecting and rebuilding is based on the polyhedral model. Therefore, the proposed compilation framework can be extended to work with other frontends, as long as a polyhedral representation can be acquired. The extension of the proposed framework is left as future work.

With the above-mentioned polyhedral tools and libraries, the polyhedral extraction and the output code generation can be performed reliably. In the next subsections, we focus on describing the other two processes: operator detection and schedule tree rebuilding.

3.2 Operator Detection

The basic idea of operator detection is subtree matching. An internal description of the target operator is built before performing the matching, including the subtree structure and the memory access relations. The structure description defines the node type of the subtree nodes and the topological relations between them. The memory access relations describe the accessed array indices by each statement instance. For example, the write access relation of an MM should be an affine relation:

$$(i, j, k) \rightarrow (i, j), \quad (1)$$

which means statement instances with indices (i, j, k) should access the array element with indices (i, j) .

Then the schedule tree of the original source code is searched in a bottom-up manner to find the subtrees that have the same structure as the target operator. When a structure match is found, both the read and write access relations are compared with the above-mentioned pre-built relations. Afterward, the operator type is checked to ensure that it is a **multiply-and-accumulate (MAC)** statement.

Based on the detection approach described above, we propose to increase the types of operators to achieve better detection in NN workloads. First, we implement the detection of the CONV operator. From Listing 2, we can see that, although CONV has a similar subtree structure as MM, the dimensionality is higher, and the access relations are more complicated. Therefore, dedicated processing for CONV is necessary. In addition, pooling is also an import class of NN operators and

Table 1. Supported Operators

Order	Operator
1	CONV+INIT+ACT
2	CONV+ACT
3	CONV+INIT
4	Bare CONV
5	Pooling
6	MM+INIT+ACT
7	MM+ACT
8	MM+INIT
9	Bare MM
10	MV+INIT+ACT
11	MV+ACT
12	MV+INIT
13	Bare MV

```

/* Output channel */
for (j = 0; j < M; j++)
  /* Output feature map X */
  for (k = 0; k < E; k++)
    /* Output feature map Y */
    for (m = 0; m < F; m++) {
S1:      Ofmap[i][j][k][m] = 0.0;
        /* Input channel */
        for (n = 0; n < C; n++)
          /* Input feature map X */
          for (p = 0; p < R; p++)
            /* Input feature map Y */
            for (q = 0; q < S; q++)
S2:      Ofmap[i][j][k][m] +=
          Filter[j][n][p][q] *
          Ifmap[i][n][k+p][m+q];
    }

```

Listing 2. Example code of CONV.

can be viewed as a variation of CONV, where the number of input and output channels are the same, and the statement is a call to pooling function instead of a MAC.

Second, as an important class of operators in NN applications, activation functions are implemented by hardware in memristor-based accelerators. These activation functions are either directly fused with MM, or tightly coupled and computed at the same location with MM. Therefore, instead of detecting activation functions alone, we implement the detection of the fused operator of MM/CONV and activation.

In addition, we distinguish the detection of operators with initialization and bare ones where initialization statements (e.g., S1 in Listing 2) are not included.

The detection of different operators is carried out sequentially. All the supported operators and the order of detecting are listed in Table 1. Fused operators have priority over bare counterparts, and operators with higher dimensions precede those with lower dimensions.

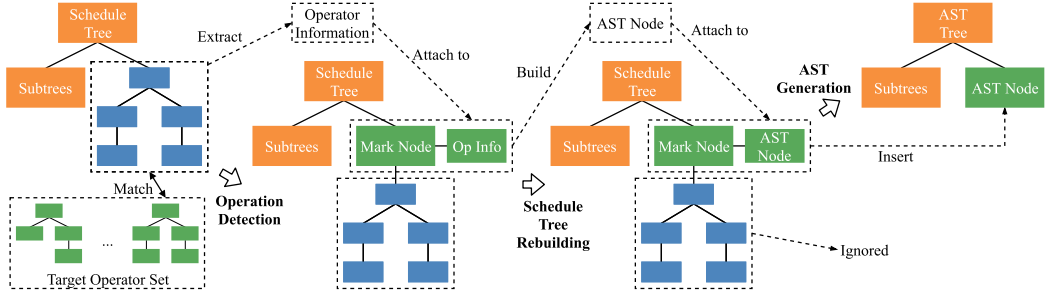


Fig. 4. Operator detection, schedule tree rebuilding, and AST generation.

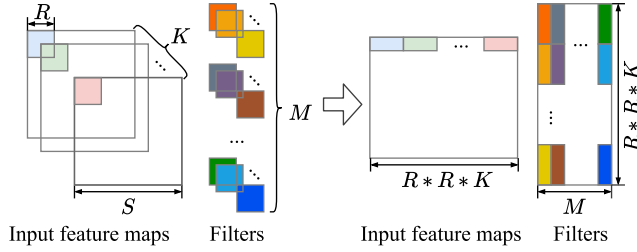


Fig. 5. Illustration of convolution mapping. R is filter size; S is input feature size; C is the number of input channels; M is the number of output channels.

When a match is found, a mark node is inserted into the schedule tree, as shown in Figure 4. The inserted mark node is used to mark the subtree as a detected operator. It is also attached with information for AST generation in the next phase. That includes the partial iteration domain of the operator, identifiers of the accessed arrays, and so forth.

3.3 Schedule Tree Rebuilding

During the rebuilding phase, the inserted mark nodes and the attached information in the matching phase are used to rebuild the schedule tree. As mentioned above, we directly build the AST node that will be inserted into the generated AST in place of the original node. The basic idea is to build an AST node that generates a call to the corresponding API function. The built AST node is also attached to the mark node in the schedule tree. Finally, during the AST generation, the schedule tree is traversed in a top-down manner to build the output AST tree. When a mark node is encountered, the AST node is inserted into the result tree, and the subtree of the mark node is ignored. This process is illustrated in Figure 4.

Considering the practical accelerator design, the rebuilding should handle the following issues.

Transformation of CONV: CONV operators are converted to MM with the mapping scheme illustrated in Figure 5. Each 2-D kernel of $R \times R$ is expanded into a 1-D dot product of $R \times R$. Based on that, the CONV kernels can be converted into a weight matrix of $(R \times R \times K) \times M$, where K is the number of input channels, and M is the number of output channels. The shape of the input feature maps is adjusted accordingly. To perform such data layout transformation, API functions are inserted before and after the MM function to load and store data.

Operator tiling: Another concern is that MM operations need to be tiled due to the limited processing size of crossbar-based PEs, as shown in Figure 6. Therefore, the result AST node is a

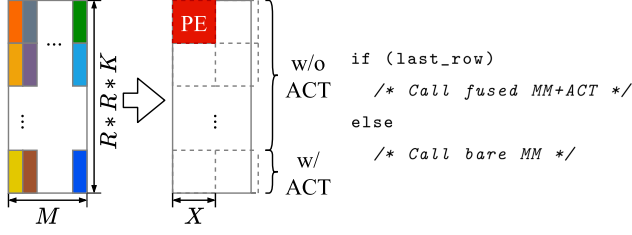


Fig. 6. Tiling among PEs. X is the processing size of PE.

```

/* Before tiling */
for (int i=0; i<N; i++)
  for (int j=0; j<N; j++)
    for (int k=0; k<N; k++)
      /* MAC operation */;
/* After tiling */
for (int ii=0; ii<N; ii+=T)
  for (int jj=0; jj<N; jj+=T)
    for (int kk=0; kk<N; kk+=T)
      for (int i=0; i<T; i++)
        for (int j=0; j<T; j++)
          for (int k=0; k<T; k++)
            /* MAC operation */;
/* After generation */
for (int ii=0; ii<N; ii+=T)
  for (int jj=0; jj<N; jj+=T)
    for (int kk=0; kk<N; kk+=T)
      /* Generated function call that
       implements MM of size T*T*T
       with memristor crossbar. */

```

Listing 3. Example code of MM before tiling, after tiling, and after code generation.

for node that contains the outer loops of the tiled loop nest, whereas the inner loops are carried out by the generated function call, as shown in Listing 3.

Fused operators: When dealing with the fused operator of MM and activation, the activation should be carried out only after the final accumulation, as shown in Figure 6. The partial results from PEs in the same column are accumulated and then go through the activation unit. Therefore, we generate a guarding *if* node to select whether a bare MM or a fused MM activation is called according to the loop index.

Invocation granularity: According to specific usage scenarios, the proposed framework can be configured to generate different types of AST nodes in different invocation modes:

- **Coarse-grained invocation:** In some cases, the accelerator may be equipped with its customized toolchain. The host can invoke the accelerator with coarse-grained function calls which simply give the operands and the iteration domains to the toolchain. Then the function calls are lowered to the configuration of the accelerator by its toolchain. An example is FPSA [24], which has its toolchain to perform neural synthesis, placement, and routing, similar to the conventional flow of **field-programmable gate arrays (FPGAs)**. Therefore,

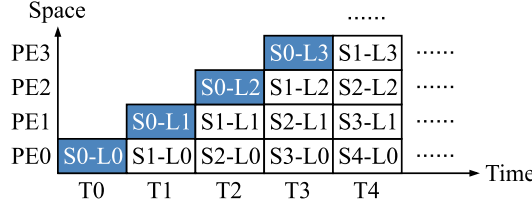


Fig. 7. Pipeline example with four PEs. S_0 – S_4 are five samples in one batch.

in the coarse-grained invocation mode, the AST node is built as a *user* node that contains only one function call.

- Fine-grained invocation: In the fine-grained invocation mode, the proposed framework controls details of execution. Therefore, in contrast to the coarse-grained mode, all the above-mentioned factors are properly handled by the framework.

3.4 Support for Pipeline Generation

In NN workloads, layers have data dependence on their predecessors, which limits the processing parallelism when these layers are mapped to different regions of the accelerator. Therefore, the compilation framework generates a pipelined execution code to improve processing throughput.

The pipeline makes use of the *batch* and *layer* dimension of NN workloads, as shown in Figure 7. Note that several PEs may be used to accommodate the weights of one large layer if necessary. We use one PE for one layer for simplicity in Figure 7. The PEs in the accelerator are assigned to process a certain layer of the NN. For example, at time T_0 , only layer L_0 of sample S_0 can be processed with PE_0 , and at time T_1 , PE_1 starts processing layer L_1 of sample S_1 and PE_0 starts processing layer L_0 of S_1 . In such a manner, all the PEs stay busy after the pipeline starts.

In the rebuilding phase, before performing any rebuilding, the proposed framework first analyzes all the detected operators to decide whether they can be formed as a pipeline. Then the proposed framework allocates the PEs among the operators to maximize the pipeline throughput. This process is different from previous polyhedral-based compilers [15, 37], which generate the offloading code right after the detection of one operator and continue to process other operators with a similar method. This leads to performance loss when large batches are available since pipelining reduces the idle time of PEs significantly. As shown in Section 4.4.5, this performance loss can be 10× and 27× for the CNN benchmarks with a batch size of 64.

3.4.1 Resource Allocation. Given the total number of available PEs, the proposed compilation framework allocates the hardware resource among the operators. Since different operators reuse their weights for different times, more PEs should be assigned to those operators that reuse weights more times. By replicating the weights of these operators, their execution latency can be reduced proportionally. We referred to this number of times of replication as *replication degree*. Therefore, the center of resource allocation is to decide the replication degree of each operator.

Algorithm 1 is used to find the replication degrees with the total number of PEs given. At the beginning of the procedure, the *replication degree* of each layer is set to be one, which means that no weights are replicated. Then the replication degrees are updated iteratively. In each iteration, we find all the *bottleneck* layers that incur the maximum latency and check whether the remaining number of PEs can accommodate one more copy of the weights of bottleneck layers. If so, the corresponding replication degrees are incremented so that the bottlenecks are optimized, and the algorithm goes back to re-identify the bottleneck layers; otherwise, no further optimization can be made, and the procedure ends.

ALGORITHM 1: Balanced resource allocation

Input: N : total # of PEs
 $n[i]$: min # of PEs required by layer i
 $reuse[i]$: total reuse times of layer i
 L : total # of layers

Output: $degree[i]$: replication degree of layer i

```

1:  $degree[i] \leftarrow 1$  for  $i \leftarrow 0$  to  $L - 1$ 
2: while true do
3:    $latency[i] \leftarrow \lceil (reuse[i] / degree[i]) \rceil$  for  $i \leftarrow 0$  to  $L - 1$ 
4:    $maximum \leftarrow \{i | latency[i] = \max(latency)\}$ 
5:    $increment \leftarrow \sum n[i]$  for  $i$  in  $maximum$ 
6:    $used \leftarrow \sum (n[i] * degree[i])$  for  $i \leftarrow 0$  to  $L - 1$ 
7:   if  $increment + used > N$  then
8:     return  $degree[i]$  for  $i \leftarrow 0$  to  $L - 1$ 
9:   end if
10:   $degree[i] \leftarrow degree[i] + 1$  for  $i$  in  $maximum$ 
11: end while

```

With the allocation scheme decided, the proposed framework generates corresponding PE indices that will be passed to the API function calls as arguments, as shown in Listing 4.

3.4.2 Pipeline Code Generation. The generated code describes the execution order illustrated in Figure 7. In the original schedule tree, the batch dimension is identified as the loop that surrounds a sequence of target operators. In the rebuilding phase, the iteration domain of this loop is modified from $\{i : 0 \leq i \leq B\}$ to $\{i : 0 \leq i \leq B + L - 1\}$, where B is the batch size, and L is the number of layers. In the generated code, the corresponding function for layer L_N is called only when i is in the range of $N \leq i \leq N + B - 1$. In this way, the pipeline code can be generated.

3.4.3 Runtime Support. Some runtime components are required to enable a non-blocking of-flooding to support the above-mentioned pipelined execution. For example, at time T_1 , the host should offload both layer L_1 of S_0 and layer L_0 of S_1 to the accelerator. This means that after the host offloads layer L_N to the accelerator with a function call, it should be able to execute the function call for layer L_{N+1} before the computation is finished.

This can be achieved by a priority queue with a controller that manages the availability of PEs and the scheduling of in-queue offloading tasks. The host offloads the operators to this queue and stops offloading when synchronization is required. Then the offloading queue offloads the stalest candidate whose target PE is available in each cycle.

To facilitate such an execution manner, the proposed compilation framework inserts fence function at the end of each timestep.

3.5 Compilation Arguments

To work with the above workflow, the proposed compilation framework accepts some compilation arguments to specify the hardware constraints.

One argument is the total number of PEs. This number is used during the invocation of Algorithm 1 as one of its inputs. Another important constraint is the dimensions of supported operators such as MV and MM. Two dimensions are required for MV generation, while three are required for MM. Note that the dimension constraints of matrix operators are decided by two factors, the aggregated size of memristor crossbars and the total buffer size inside PE. In most accelerator

Table 2. Synthetic Kernels

Kernel	Description	# Matches
mv-bare	Bare MV	1/1
mv-init	MV with initialization	1/1
mm-bare	Bare MM	1/1
mm-init	MM with initialization	1/1
mm-act	MM with activation	1/1
batch-mm	Batched MM	1/1
conv-bare	Bare CONV	1/1
conv-init	CONV with initialization	1/1
conv-act	CONV with activation	1/1
batch-conv	Batched CONV	1/1
mlp-4	Four-layer MLP with activation	3/3*
conv-2	Two consecutive CONV layers	2/2

*A four-layer MLP contains three MMs.

Table 3. NN Benchmarks

	# CONV Layers	# FC Layers	# Weights
VGG-16	13	3	138M
ResNet-50	53	1	25.5M
MLP-M	0	4	1.4M
MLP-L	0	4	3.2M

designs, these two parameters are co-designed so that neither one imposes a stricter constraint than the other, to avoid inefficient resource utilization [12, 18, 33, 35].

Besides these compilation arguments, the compilation framework also accepts some control arguments, which decide whether to use coarse-grained invocation and whether to enable pipeline generation.

4 EVALUATION

4.1 Experimental Setup

4.1.1 Benchmarks. We use two sets of benchmarks in our evaluation. One is a set of synthetic kernels, each composed of one or more operators of interest. The kernels are listed in the first column in Table 2.

In addition, we use two MLPs from [12] (denoted as MLP-M and MLP-L) and two CNNs, VGG-16 [34] and ResNet-50 [20] to evaluate the performance of the generated code. The details of these benchmarks are summarized in Table 3.

Based on these benchmarks, we test the detection and generation capability of the proposed compilation framework. We also evaluate the generated code to verify its correctness. The results are shown in Sections 4.2 and 4.3.

4.1.2 Memristor-Based Architecture. To evaluate the proposed compilation framework, we choose two memristor-based architectures, ISAAC [33] and FPSA [24], and use them as two usage cases of the proposed framework. Specifically, in the ISAAC case, we use the fine-grained invocation mode, where the accelerator provides only MM and MM with activation functionality. The mapping of CONV operators and the generation of the pipeline are controlled by the proposed

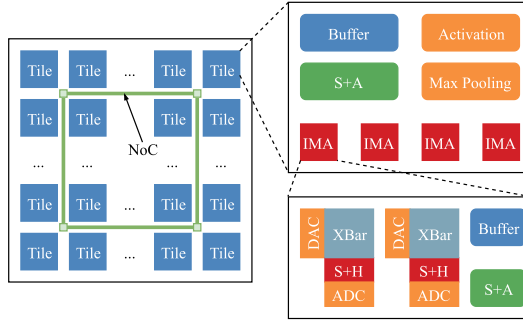


Fig. 8. ISAAC architecture [33].

compilation framework. Whereas in the FPSA case, due to the feature of FPSA's hardware implementation, the coarse-grained invocation mode is used. The software stack of FPSA is involved after the proposed compiler generates coarse-grained function calls. Details are discussed in Sections 4.4 and 4.5.

Since the support of pipeline is a critical difference between the proposed framework with previous studies [15, 37], we also evaluate the impact of pipelined execution within the ISAAC case, as shown in Section 4.4.5.

4.2 Detection and Generation

The last column in Table 2 lists the number of target operators and the number of matches found in each kernel. As shown in the table, the proposed compilation framework succeeds in detecting all the operators in the synthetic kernels, which indicates that it can reliably detect for most cases in NN workloads. As discussed in Section 3, the usage of multiple detection passes provides good coverage on different types of operators. For example, the *mlp-4* kernel is composed of two MM with activation and one without activation. The former are captured first during the detection for fused MM, while the latter is detected as a bare MM operator. Note that a similar workload (referred to as *mlp-3*) was used in the evaluation of TC-CIM [15], and TC-CIM failed to detect the final MM because its matcher can only match MM operators with an initialization statement.

4.3 Correctness

Since the proposed compilation framework directly builds the AST node, we also verified the correctness of the generated code. The verification is done by implementing the API functions with a behavior model and comparing the computation results with the original code. The verification of all the synthetic kernels and the NN benchmarks passed without introducing any errors, which indicates that the rebuilding phase is reliable. One important reason is the regularity of the iteration domain and the dependence relation of NN operators.

4.4 Case Study I: ISAAC

4.4.1 Architecture and Configuration. The architecture of ISAAC [33] is shown in Figure 8. It contains 168 tiles per chip, and each tile has 12 *in-situ multiply-accumulate* (IMA) units and other functional units such as activation units and input/output registers. The IMA unit contains eight 128×128 memristor crossbars. We adopt the configuration in [33], where eight 2-bit memristor cells in one row are used to represent a 16-bit weight. Therefore, each tile can calculate a MM with a size of 1536×128 . These constraints are inputted to the proposed compiler so that it can tile the operators into proper sizes.

```

/* MV */
void xb_mv_bare(int m, int n, uint16_t *A, int lda, uint16_t *B,
               uint16_t *C, int init, int pos);
void xb_mv_act(int m, int n, uint16_t *A, int lda, uint16_t *B,
               uint16_t *C, int init, int act_type, int pos);
/* MM */
void xb_mm_bare(int m, int n, int k, uint16_t *A, int lda, uint16_t *B, int ldb,
               uint16_t *C, int ldc, int init, int pos);
void xb_mm_act(int m, int n, int k, uint16_t *A, int lda, uint16_t *B, int ldb,
               uint16_t *C, int ldc, int init, int act_type, int pos);
/* Memory allocation */
void xb_malloc_2d(uint16_t *A, int d0, int d1, int pos);
void xb_malloc_3d(uint16_t *A, int d0, int d1, int d2, int pos);
/* Data layout transformation */
void conv_load_filter(uint16_t *src, uint16_t *dst, int d0,
                     int d1, int d2, int d3);
void conv_load_input(uint16_t *src, uint16_t *dst, int din_0, int din_1,
                    int din_2, int din_3, int dout_0, int dout_1, int pos);
void conv_store_output(uint16_t *src, uint16_t *dst, int din_0, int din_1,
                      int dout_0, int dout_1, int dout_2, int pos);

```

Listing 4. Programming interface functions.

Multiple ISAAC chips can be connected to accommodate the weights of large NN models. In our experiment, we evaluate the performance with 1 to 8 chips for the MLP benchmarks, and with 8 to 64 chips for the CNN benchmarks.

4.4.2 Programming Interface. We build a BLAS-like API for the target architecture, as listed in Listing 4. Since ISAAC uses 16-bit numbers to represent weights and input/output data, data type `uint16_t` is used in the API. The API includes functions that can offload MV/MM with the above-mentioned size. It also includes functions that allocate memory space in the accelerator. Since no direct CONV is supported, two functions, `conv_load_input` and `conv_store_output`, are designed to carry out the data layout transformation. The argument `pos` is used to specify the index of PE to support the resource allocation and pipelined execution, as discussed in Section 3.4.

4.4.3 Methodology. With the generated code, the mapping and execution order of each operator can be decided. The performance overhead of both on-chip and off-chip data exchange can be estimated statically. Therefore, the performance for a given NN workload can be expressed with an analytical model. A similar method is also employed in [33].

We use the metric *Computation Density* (measured by number of multiplication/addition operations performed per second per square millimeter, GOPs/s/mm^2) for the performance evaluation. The area term is introduced to make a normalized comparison between configurations with different numbers of chips.

4.4.4 Performance. Figure 9 shows the results of the computation density for the four benchmarks with different batch sizes and numbers of chips. For all the benchmarks, the computation density improves as the batch size increases. This is because more samples make better utilization of the pipeline to hide latency. For the MLP benchmarks, larger batch size also contributes to higher hardware utilization as they feature small matrix sizes. With a large batch size, the weights are replicated to occupy more PEs on the chip.

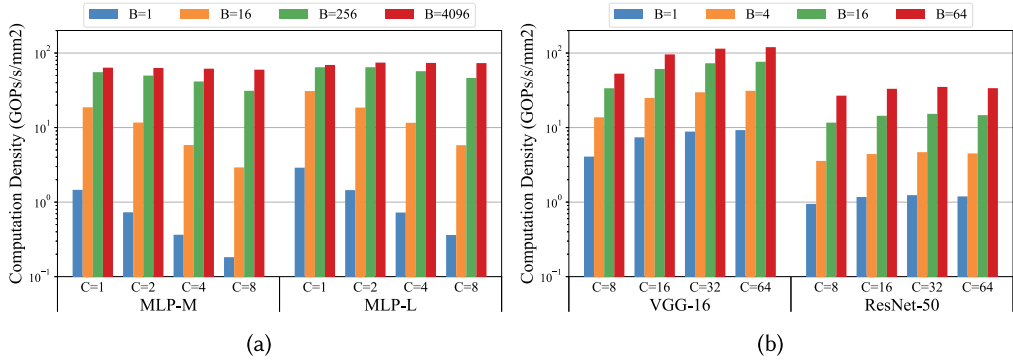


Fig. 9. Computation density with different batch size (B) and number of chips (C) for (a) MLPs and (b) CNNs.

For the CNN benchmarks, the computation density improves with more chips available. The reason is that the first few layers in those CNNs require a large replication degree to make a balanced pipeline. When more chips are available, these bottleneck layers obtain more hardware resources, which improves the performance of the whole pipeline. Since such replication is not required by MLPs, the MLP benchmarks see little improvement when the number of chips increases.

As the available number of chips increases (C in Figure 9), the computation density results have a consistent trend with those in [33] where benchmarks are manually mapped onto the accelerator. These results demonstrate the efficiency of the proposed framework to exploit the on-chip hardware resources and the capability of conducting reliable transparent offloading.

Furthermore, we use a synthetic CNN layer that can perfectly match the PEs for validation with peak results reported in ISAAC. The computation density result is 352 GOPs/s/mm², while the reported result is 479 GOPs/s/mm². Our evaluation result is lower because PEs are idle during memory accesses; if we only consider the busy cycles, the computation density will be 483 GOPs/s/mm², which matches the reported computation density by ISAAC closely. From another perspective, all the computation density results of the four NN benchmarks can be viewed as a combination of peak computation density with average utilization. As stated in [33], the utilization problem is unavoidable even in manual mapping. Although the first few layers of the CNN benchmarks are replicated, the overall performance still suffers from the imbalanced pipeline and the low temporal utilization in the last few layers. In addition, the size mismatch between actual MMs and PEs also leads to lower spatial utilization than the ideal case. These factors cause the gap between the actual and peak performance reported by ISAAC.

As shown in Figure 10, we vary the size of memristor crossbars and evaluate the performance while keeping the total storage amount unchanged. On the one hand, using smaller crossbars gives a larger number of PEs, which provides more flexibility for mapping; thus, the imbalance of the pipeline can be alleviated. Using smaller crossbars also contributes to improving spatial utilization. On the other hand, with the total storage amount unchanged, using smaller crossbars requires larger hardware overhead since an increase of peripheral circuits is needed.

Figure 10 shows the results: Despite the increase in hardware overhead, the computation density is higher with smaller crossbar sizes. Note that in [33], their design space exploration decided that using crossbars of size 128×128 gives higher peak computation density compared with that using crossbars of size 64×64 . The main reason behind this difference is the gap in actual crossbar utilization. These results indicate that providing flexibility for mapping is a critical design consideration. In addition, integrating multiple crossbars with different sizes instead of monolithic design has also been investigated [47].

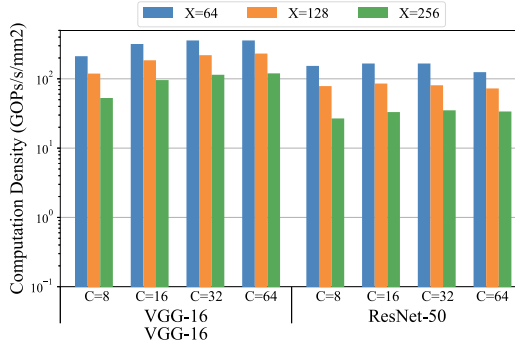


Fig. 10. Computation density with different crossbar size (X) and number of chips (C) for CNNs.

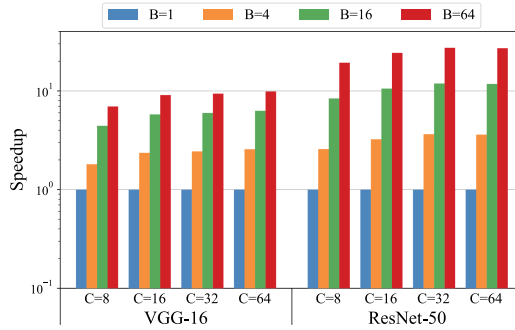


Fig. 11. Impact of pipelined execution: Speedup relative to non-pipelined execution.

4.4.5 Impact of Pipeline. We evaluate the impact of pipelined execution on the CNN benchmarks, compared with the performance when the pipeline is not enabled. The same number of PEs are used in the two scenarios, and the same allocation decided by Algorithm 1 is employed. The speedup results are shown in Figure 11. When the batch size is one, there is no pipelined execution even with the pipeline feature enabled since the layers are executed sequentially, which leads to a speedup of one. When the batch size is more than one, the speedup increases as the batch size increases, since the total execution time increases marginally when the pipeline is enabled, whereas the execution time increases linearly when the pipeline is not used. Specifically, for VGG-16, the speedup is 1.8× to 2.6×, 4.4× to 6.3×, and 7.0× to 9.9× with a batch size of 4, 16, and 64, respectively. Similar trends are observed for ResNet-50, with speedup ranging from 2.6× to 27.1×.

Figure 12 shows the impact of the optimized allocation decided by Algorithm 1 when the pipeline generation is enabled. We use an average allocation as the baseline, where all the PEs are allocated evenly between all the layers. In both scenarios, the pipelined execution is enabled, and the same number of PEs are used so that the speedup results are the same as energy efficiency results. It can be observed that the allocation algorithm has more impact on performance when more PEs are available since the algorithm ensures that the increased PEs are used to alleviate the performance bottleneck. For VGG-16, the speedup is 1.6× to 3.6×, and for ResNet-50, the speedup is 3.9× to 5.1×.

These results show the importance of optimizing the NN workloads from the network perspective. Failing to exploit such optimization opportunities will result in failing to fully exploit

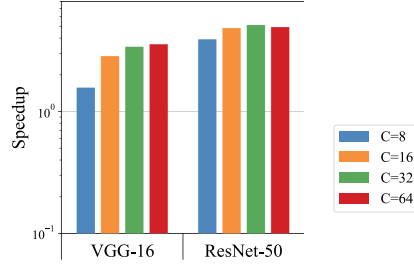


Fig. 12. Impact of optimized allocation: Speedup relative to average allocation.

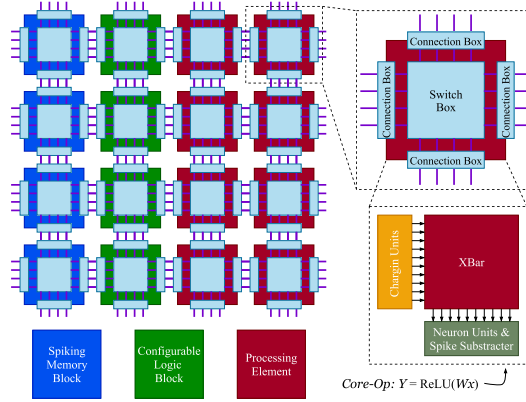


Fig. 13. FPSA [24] architecture.

the efficiency of memristor-based accelerators. In contrast to previous studies [15, 37], the proposed framework can make such optimization by detecting and optimizing all the operators before rebuilding.

4.5 Case Study II: FPSA

4.5.1 Architecture and Methodology. Figure 13 shows an overview of FPSA architecture [24]. It contains three kinds of function blocks: spiking memory block for buffering, configurable logic block for controlling, and PE for computation. In its PE implementation, a spiking schema is used to avoid the high overhead of DAC and ADC. FPSA also features novel peripheral units, including charging units, neuron units, and spike subtractors. With these peripheral units, PEs in FPSA provide an atomic implementation of $Y = ReLU(Wx + b)$ as its *core-op*. Therefore, it depends on its *neural synthesizer* [23] to convert NN workloads into models that are represented with its core-op, which can be mapped to its PEs.

This feature makes the fine-grained invocation mode not suitable for FPSA. Instead, we use the coarse-grained invocation of the proposed framework to generate function calls without changing the operators but leave them for the toolchain of FPSA. We adopted the hardware configuration in [24] and used these tools to evaluate the critical path and the hardware overhead.

FPSA's resource allocation uses *duplication degree* to denote the maximum duplication number of core-ops for each layer, which is similar to the replication degree used in the proposed framework. In FPSA's toolchain, the duplication degree is specified, and then the toolchain does the synthesis and place-and-route and computes the actual performance and area of the workload.

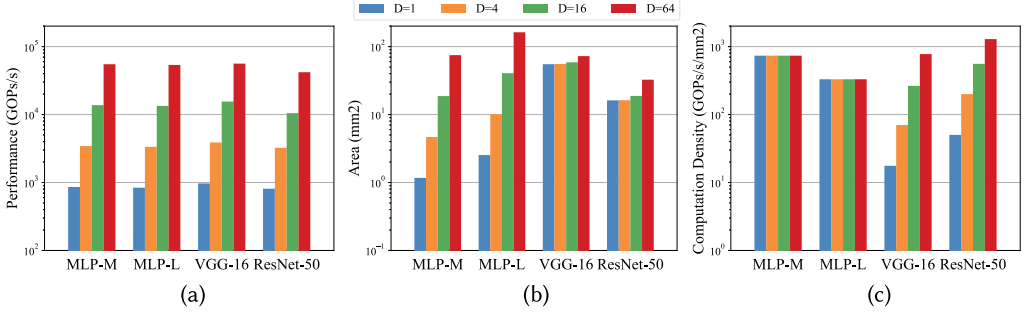


Fig. 14. Evaluation results with different duplication degrees (D): (a) Performance; (b) Area; (c) Computation density.

This is similar to the conventional workflow of FPGAs. In the experiment, we evaluate the performance and hardware overhead with duplication degrees of 1/4/16/64. To work with FPSA's tools, we implemented a tool that can convert the generated code to the input of the tools.

4.5.2 Performance. As shown in Figure 14, the performance for MLPs increases proportionally as the duplication degree increases. This is because weights in MLP are not reused, and duplication can directly result in an increase in performance. For the same reason, the area overhead for MLPs shows a similar trend of increasing as the duplication degree increases, which leads to a constant computation density with different duplication degrees. For CNNs, the performance results are similar to those of MLPs, whereas the area overhead increases much slower than that for MLPs. As mentioned above, to balance the pipeline of CNN processing, the first few layers require larger duplication degrees than the last few layers. When duplication degree increases, only the first few layers are replicated as needed, which leads to a moderate increasing in the area. As a result, the computation density improves as the duplication degree increases. These results are consistent with those in [24], which indicates the reliability of the proposed compilation framework.

5 DISCUSSION

5.1 Target Architecture

As mentioned above, some studies proposed ISA and corresponding compilers for memristor-based NN accelerators. Compared with the target architecture of the proposed compilation framework, where accelerators are controlled by host processors, these accelerators require hardware units for instruction fetching and decoding. Implementing such hardware units usually lowers the area and energy efficiency of memristor crossbars. For example, PUMA [5] has a lower area and energy efficiency than ISAAC [33], due to the overhead to achieve better programmability. Another ISA-based design [16] reduces the overhead by not supporting control transfer instructions, and the compiler is responsible to carry out loop unrolling.

On the other hand, in our target architecture, the execution of loops is controlled by the host processor, and thus the design of accelerators can be simplified. With the proposed compilation framework, this work contributes to improving the programmability of such architecture and keeping its simple hardware implementation at the same time.

5.2 Support for NN Training

The proposed compilation framework only supports the compilation of NN inference workloads, as a majority of memristor-based NN accelerators focus on inference [28]. Since training is also

important in NN processing, some studies propose to use memristor-based architectures to accelerate NN training [4, 11, 35]. Some of them also employ API-based interfaces, which is similar to the target architecture of the proposed compilation frameworks. Since the main body of the backward process in NN training is also composed of operators that are similar to the detection targets in the proposed compilation framework, it is feasible for the proposed compilation framework to support such training accelerators with proper extension.

One critical challenge is that different training accelerators use different strategies to optimize the memristor writes brought by weight updating. For example, Pipelayer [35] computes partial derivatives with memristor crossbars and updates weights by reading them out, merging them with partial derivatives, and writing updated weights back. Another accelerator design, Panther [4], updates weights with outer product accumulate operations, where input voltages are fed to memristor crossbars' bitlines and wordlines simultaneously. An observation is that different updating strategies require different model mappings and execution flows. The compilation framework needs to consider such strategies and conduct proper transformations to cooperate with them. We leave such extension as future work.

6 CONCLUSION

In this article, we proposed a polyhedral-based compilation framework for memristor-based NN accelerators. With the extension of the target NN operator set, the framework is capable of processing whole NN workloads, and different invocation granularities are designed to support various architectures. The proposed compilation framework also implements support for pipeline, including an overall consideration of resource allocation and operator tiling based on hardware constraints such as crossbar sizes. By conducting case studies on two typical memristor-based architectures, we demonstrated the generality of the proposed framework over accelerators with different architectural designs. Based on the evaluation results, we give analyses on the performance gap between compilation results and peak results from the perspective of hardware utilization.

REFERENCES

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek Gordon Murray, Benoit Steiner, Paul A. Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI'16)*. USENIX Association, 265–283.
- [2] Joao Ambrosi, Aayush Ankit, Rodrigo Antunes, Sai Rahul Chalamalasetti, Soumitra Chatterjee, Izzat El Hajj, Guilherme Fachini, Paolo Faraboschi, Martin Foltin, Sitao Huang, Wen-Mei Hwu, Gustavo Knappe, Sunil Vishwanathpur Lakshminarasimha, Dejan S. Milojicic, Mohan Parthasarathy, Filipe Ribeiro, Lucas Rosa, Kaushik Roy, Plinio Silveira, and John Paul Strachan. 2018. Hardware-software co-design for an analog-digital accelerator for machine learning. In *2018 IEEE International Conference on Rebooting Computing (ICRC'18)*. IEEE, 1–13. <https://doi.org/10.1109/ICRC.2018.8638612>
- [3] Dario Amodei, Sundaram Ananthanarayanan, Rishita Anubhai, Jingliang Bai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, Erich Elsen, Jesse H. Engel, Linxi Fan, Christopher Fougner, Awni Y. Hannun, Billy Jun, Tony Han, Patrick LeGresley, Xiangang Li, Libby Lin, Sharan Narang, Andrew Y. Ng, Sherjil Ozair, Ryan Prenger, Sheng Qian, Jonathan Raiman, Sanjeev Satheesh, David Seetapun, Shubho Sengupta, Chong Wang, Yi Wang, Zhiqian Wang, Bo Xiao, Yan Xie, Dani Yogatama, Jun Zhan, and Zhenyao Zhu. 2016. Deep speech 2: End-to-end speech recognition in English and Mandarin. In *Proceedings of the 33rd International Conference on Machine Learning (ICML'16) (JMLR Workshop and Conference Proceedings)*, Vol. 48. JMLR.org, 173–182.
- [4] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Sapan Agarwal, Matthew J. Marinella, Martin Foltin, John Paul Strachan, Dejan S. Milojicic, Wen-Mei Hwu, and Kaushik Roy. 2020. PANTHER: A programmable architecture for neural network training harnessing energy-efficient ReRAM. *IEEE Trans. Comput.* 69, 8 (2020), 1128–1142. <https://doi.org/10.1109/TC.2020.2998456>

- [5] Aayush Ankit, Izzat El Hajj, Sai Rahul Chalamalasetti, Geoffrey Ndu, Martin Foltin, R. Stanley Williams, Paolo Fara-boschi, Wen-mei W. Hwu, John Paul Strachan, Kaushik Roy, and Dejan S. Milojevic. 2019. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)*. ACM, 715–731. <https://doi.org/10.1145/3297858.3304049>
- [6] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman P. Amarasinghe. 2019. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO'19)*. IEEE, 193–205. <https://doi.org/10.1109/CGO.2019.8661197>
- [7] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A practical automatic polyhedral paral-lelizer and locality optimizer. In *Proceedings of the ACM SIGPLAN 2008 Conference on Programming Language Design and Implementation*. ACM, 101–113. <https://doi.org/10.1145/1375581.1375595>
- [8] Lorenzo Chelini, Oleksandr Zinenko, Tobias Grosser, and Henk Corporaal. 2020. Declarative loop tactics for domain-specific optimization. *ACM Trans. Archit. Code Optim.* 16, 4 (2020), 55:1–55:25. <https://doi.org/10.1145/3372266>
- [9] Yu-Hsin Chen, Joel S. Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA'16)*. IEEE Computer Society, 367–379. <https://doi.org/10.1109/ISCA.2016.40>
- [10] Yunji Chen, Tao Luo, Shaoli Liu, Shijin Zhang, Liqiang He, Jia Wang, Ling Li, Tianshi Chen, Zhiwei Xu, Ninghui Sun, and Olivier Temam. 2014. DaDianNao: A machine-learning supercomputer. In *47th Annual IEEE/ACM Interna-tional Symposium on Microarchitecture (MICRO'14)*. IEEE Computer Society, 609–622. <https://doi.org/10.1109/MICRO.2014.58>
- [11] Ming Cheng, Lixue Xia, Zhenhua Zhu, Yi Cai, Yuan Xie, Yu Wang, and Huazhong Yang. 2017. TIME: A training-in-memory architecture for memristor-based deep neural networks. In *Proceedings of the 54th Annual Design Automation Conference (DAC'17)*. ACM, 26:1–26:6. <https://doi.org/10.1145/3061639.3062326>
- [12] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A novel processing-in-memory architecture for neural network computation in ReRAM-Based main memory. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA'16)*. IEEE Computer Society, 27–39. <https://doi.org/10.1109/ISCA.2016.13>
- [13] Lei Deng, Yuan Xie, Ling Liang, Guanrui Wang, Liang Chang, Xing Hu, Xin Ma, Liu Liu, Jing Pei, and Guoqi Li. 2020. SemiMap: A semi-folded convolution mapping for speed-overhead balance on crossbars. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 39, 1 (2020), 117–130. <https://doi.org/10.1109/TCAD.2018.2883959>
- [14] Xiangyu Dong, Cong Xu, Yuan Xie, and Norman P. Jouppi. 2012. NVSim: A circuit-level performance, energy, and area model for emerging nonvolatile memory. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 31, 7 (2012), 994–1007. <https://doi.org/10.1109/TCAD.2012.2185930>
- [15] Andi Drebes, Lorenzo Chelini, Oleksandr Zinenko, Albert Cohen, Henk Corporaal, Tobias Grosser, Kanishkan Vadivel, and Nicolas Vasilache. 2020. TC-CIM: Empowering tensor comprehensions for computing-in-memory. In *IMPACT 2020 - 10th International Workshop on Polyhedral Compilation Techniques*. 1–12.
- [16] Daichi Fujiki, Scott A. Mahlke, and Reetuparna Das. 2018. In-memory data parallel processor. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'18)*. ACM, 1–14. <https://doi.org/10.1145/3173162.3173171>
- [17] Tobias Grosser, Hongbin Zheng, Raghu Aloor, Andreas Simbürger, Armin Größlinger, and Louis-Noël Pouchet. 2011. Polly-polyhedral optimization in LLVM. In *Proceedings of the 1st International Workshop on Polyhedral Compilation Techniques (IMPACT)*, Vol. 2011. 1–6.
- [18] Jianhui Han, He Liu, Mingyu Wang, Zhaolin Li, and Youhui Zhang. 2020. ERA-LSTM: An efficient ReRAM-based architecture for long short-term memory. *IEEE Trans. Parallel Distrib. Syst.* 31, 6 (2020), 1328–1342. <https://doi.org/10.1109/TPDS.2019.2962806>
- [19] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient inference engine on compressed deep neural network. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA'16)*. IEEE Computer Society, 243–254. <https://doi.org/10.1109/ISCA.2016.30>
- [20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR'16)*. IEEE Computer Society, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [21] Miao Hu, John Paul Strachan, Zhiyong Li, Emmanuelle M. Grafals, Noraica Davila, Catherine Graves, Sity Lam, Ning Ge, Jianhua Joshua Yang, and R. Stanley Williams. 2016. Dot-product engine for neuromorphic computing: Program-ming 1T1M crossbar to accelerate matrix-vector multiplication. In *Proceedings of the 53rd Annual Design Automation Conference (DAC'16)*. ACM, 19:1–19:6. <https://doi.org/10.1145/2897937.2898010>

- [22] Mohsen Imani, Saransh Gupta, Yeseong Kim, and Tajana Rosing. 2019. FloatPIM: In-memory acceleration of deep neural network training with high precision. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA'19)*. ACM, 802–815. <https://doi.org/10.1145/3307650.3322237>
- [23] Yu Ji, Youhui Zhang, Wenguang Chen, and Yuan Xie. 2018. Bridge the gap between neural networks and neuromorphic hardware with a neural network compiler. In *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'18)*. ACM, 448–460. <https://doi.org/10.1145/3173162.3173205>
- [24] Yu Ji, Youyang Zhang, Xinfeng Xie, Shuangchen Li, Peiqi Wang, Xing Hu, Youhui Zhang, and Yuan Xie. 2019. FPSA: A full system stack solution for reconfigurable ReRAM-based NN accelerator architecture. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'19)*. ACM, 733–747. <https://doi.org/10.1145/3297858.3304048>
- [25] Norman P. Jouppi, Cliff Young, Nishant Patil, David A. Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omer-nick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA'17)*. ACM, 1–12. <https://doi.org/10.1145/3079856.3080246>
- [26] Shuangchen Li, Cong Xu, Qiaosha Zou, Jishen Zhao, Yu Lu, and Yuan Xie. 2016. Pinatubo: A processing-in-memory architecture for bulk bitwise operations in emerging non-volatile memories. In *Proceedings of the 53rd Annual Design Automation Conference (DAC'16)*. ACM, 173:1–173:6. <https://doi.org/10.1145/2897937.2898064>
- [27] He Liu, Jianhui Han, and Youhui Zhang. 2019. A unified framework for training, mapping and simulation of ReRAM-based convolutional neural network acceleration. *IEEE Comput. Archit. Lett.* 18 (2019), 63–66. <https://doi.org/10.1109/LCA.2019.2908374>
- [28] Sparsh Mittal. 2019. A survey of ReRAM-based architectures for processing-in-memory and neural networks. *Mach. Learn. Knowl. Extr.* 1, 1 (2019), 75–114. <https://doi.org/10.3390/make1010005>
- [29] Ravi Teja Mullapudi, Vinay Vasista, and Uday Bondhugula. 2015. PolyMage: Automatic optimization for image processing pipelines. In *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'15)*. ACM, 429–443. <https://doi.org/10.1145/2694344.2694364>
- [30] ONNX. 2019. ONNX: Open Neural Network Exchange.
- [31] Sebastian Pop, Albert Cohen, Cédric Bastoul, Sylvain Girbal, Georges-André Silber, and Nicolas Vasilache. 2006. GRAPHITE: Polyhedral analyses and optimizations for GCC. In *Proceedings of the 2006 GCC Developers Summit*. Cite-seer, 1–18.
- [32] Ximing Qiao, Xiong Cao, Huanrui Yang, Linghao Song, and Hai Li. 2018. Atomlayer: A universal reRAM-based CNN accelerator with atomic layer computation. In *Proceedings of the 55th Annual Design Automation Conference (DAC'18)*. ACM, 103:1–103:6. <https://doi.org/10.1145/3195970.3195998>
- [33] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA'16)*. IEEE Computer Society, 14–26. <https://doi.org/10.1109/ISCA.2016.12>
- [34] Karen Simonyan and Andrew Zisserman. 2015. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations (ICLR'15)+, Conference Track Proceedings*. 1–14.
- [35] Linghao Song, Xuehai Qian, Hai Li, and Yiran Chen. 2017. PipeLayer: A pipelined ReRAM-based accelerator for deep learning. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA'17)*. IEEE Computer Society, 541–552. <https://doi.org/10.1109/HPCA.2017.55>
- [36] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014*. 3104–3112.
- [37] Kanishkan Vadivel, Lorenzo Chelini, Ali BanaGozar, Gagandeep Singh, Stefano Corda, Roel Jordans, and Henk Corporaal. 2020. TDO-CIM: Transparent detection and offloading for computation in-memory. In *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE'20)*. IEEE, 1602–1605. <https://doi.org/10.23919/DATE48585.2020.9116464>

- [38] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *CoRR* abs/1802.04730 (2018). arXiv:1802.04730.
- [39] Sven Verdoolaege. 2010. *isl*: An integer set library for the polyhedral model. In *Mathematical Software - Proceedings of the 3rd International Congress on Mathematical Software (ICMS'10)*. Lecture Notes in Computer Science, Vol. 6327. Springer, 299–302. https://doi.org/10.1007/978-3-642-15582-6_49
- [40] Sven Verdoolaege and Tobias Grosser. 2012. Polyhedral extraction tool. In *2nd International Workshop on Polyhedral Compilation Techniques (IMPACT'12)*. 1–8.
- [41] Sven Verdoolaege, Serge Guelton, Tobias Grosser, and Albert Cohen. 2014. Schedule trees. In *IMPACT - 4th Workshop on Polyhedral Compilation Techniques, Associated with HiPEAC*. ACM, 1–9.
- [42] Sven Verdoolaege, Juan Carlos Juega, Albert Cohen, José Ignacio Gómez, Christian Tenllado, and Francky Catthoor. 2013. Polyhedral parallel code generation for CUDA. *ACM Trans. Archit. Code Optim.* 9, 4 (2013), 54:1–54:23. <https://doi.org/10.1145/2400682.2400713>
- [43] Lixue Xia, Peng Gu, Boxun Li, Tianqi Tang, Xiling Yin, Wenqin Huangfu, Shimeng Yu, Yu Cao, Yu Wang, and Huazhong Yang. 2016. Technological exploration of RRAM crossbar array for matrix-vector multiplication. *J. Comput. Sci. Technol.* 31, 1 (2016), 3–19. <https://doi.org/10.1007/s11390-016-1608-8>
- [44] Peng Yao, Huaqiang Wu, Bin Gao, Jianshi Tang, Qingtian Zhang, Wenqiang Zhang, J. Joshua Yang, and He Qian. 2020. Fully hardware-implemented memristor convolutional neural network. *Nature* 577, 7792 (2020), 641–646.
- [45] Wenqiang Zhang, Xiaochen Peng, Huaqiang Wu, Bin Gao, Hu He, Youhui Zhang, Shimeng Yu, and He Qian. 2019. Design guidelines of RRAM based neural-processing-unit: A joint device-circuit-algorithm analysis. In *Proceedings of the 56th Annual Design Automation Conference 2019 (DAC'19)*. ACM, 140. <https://doi.org/10.1145/3316781.3317797>
- [46] Yongwei Zhao, Zidong Du, Qi Guo, Shaoli Liu, Ling Li, Zhiwei Xu, Tianshi Chen, and Yunji Chen. 2019. Cambricon-F: Machine learning computers with fractal von Neumann architecture. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA'19)*. ACM, 788–801. <https://doi.org/10.1145/3307650.3322226>
- [47] Zhenhua Zhu, Jilan Lin, Ming Cheng, Lixue Xia, Hanbo Sun, Xiaoming Chen, Yu Wang, and Huazhong Yang. 2018. Mixed size crossbar based RRAM CNN accelerator with overlapped mapping method. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD'18)*. ACM, 69. <https://doi.org/10.1145/3240765.3240825>

Received January 2021; revised April 2021; accepted June 2021